



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of
Computing

SEMESTER 2/20252026

SPECIAL TOPIC IN DATA ENGINEERING (SECP3843)

SECTION 01

INDIVIDUAL PROJECT ON APACHE SPARK

LECTURER:

DR ARYATI BINTI BAKRI

NAME	MATRIC NO
TEH RU QIAN	A23CS0191

Development of a Patient Visit Data Pipeline and Lakehouse Architecture Using Apache Spark

An End-to-End Analytics Platform with Docker, PostgreSQL and Metabase

Ru Qian Teh
Faculty Computing
University Teknologi Malaysia
Johor Bahru, Malaysia
tehruqian@graduate.utm.my

Abstract—Healthcare organizations collect large amounts of patient visit data every day, but the data is often stored in different systems, making it difficult to analyze and support decision-making. This project develops an end-to-end patient visit data pipeline using Apache Spark and the Data Lakehouse architecture. Data is collected from three sources: a Kaggle healthcare dataset, the WHO Global Health Observatory (GHO) API and a PostgreSQL financial database. The data is processed through the Bronze, Silver and Gold layers following the CRISP-DM methodology. After cleaning and integrating the data, it is stored in a Star Schema in PostgreSQL for analysis. An Isolation Forest model is used to detect unusual billing records, identifying 2,769 patient visits (5.04%) as anomalies. The final results are displayed in a Metabase dashboard, which includes patient demographics, admission trends, billing analysis and anomaly alerts. The completed pipeline provides a reliable platform to help hospital administrators monitor patient visits, analyze billing performance and support better decision-making.

Keywords: Patient Visit Data Pipeline, Apache Spark, Data Lakehouse, Anomaly Detection, Healthcare Analytics

I. INTRODUCTION

A. Problem Identification

Hospitals generate large numbers of patient visit data every day, including information on patient demographics, admission and discharge dates, medications, billing amounts and insurance details. This data can be analyzed to provide valuable insights for improving clinical decision-making, optimizing resource allocation and improving financial management. However, in many healthcare organizations, patient visit

data is stored in raw formats such as CSV files, making it difficult to use directly for analytical purposes.

The main problem is that patient visit data exists in isolation. They may record information about each hospital visit, but they cannot give the context that would help to determine whether the cost of treatment, length of stay or patient outcome is normal or abnormal. Further, the records are often not linked to other sources of information, such as hospital financial systems and external healthcare benchmarks. As a result, hospital administrators lack a comprehensive view of patient and financial performance, limiting their ability to monitor admission patterns, evaluate billing trends and identify potential anomalies.

The process of manually combining data from multiple sources is often time-consuming, prone to errors and difficult to scale as data volumes continue to increase. To address these challenges, this project proposes the development of an end-to-end patient visit data pipeline based on Apache Spark and a Data Lakehouse architecture. The solution combines three datasets: a healthcare dataset from Kaggle in CSV format, a dataset of health indicators from the WHO Global Health Observatory REST API and a dataset of hospital financial information stored in a PostgreSQL database. The pipeline integrates these sources into a single view

of every patient visit, allowing for better analysis from patient admission through to financial settlement.

B. Objectives

The main objective of this project is to design and implement a patient visit data pipeline.

The project objectives are:

1. To develop a data ingestion process that integrates healthcare data from CSV files, REST APIs and PostgreSQL databases into a centralized Bronze layer in Parquet format, ensuring that all source records and attributes are retained without data loss.
2. To clean, transform and enrich the healthcare data in the Silver layer by converting data types, standardizing schemas, handling missing values and deriving features to enhance data quality and value.
3. To develop Gold-layer analytical datasets modelled in a Star Schema and to visualize healthcare insights using PostgreSQL and Metabase dashboards.

II. LITERATURE REVIEW

A. The Challenge of Patient Visit Data Monitoring in Hospital Environments

In the last few years, the lack of effective patient-visit monitoring has been reported across hospital environments. According to Sekarini et al. [1] the hospitals are still using manual or semi-manual systems to record patient data for inpatient and outpatient visits, which leads to inaccurate records, bottlenecks in decision making and lack of integration between information systems. This observation is aligned with the finding by Saraswat et al. [2] that the patient health data is unstructured and distributed across multiple systems, making it impossible to extract meaningful insights without a structured automated pipeline. Both studies show that it is not just a data storage problem, it is a data accessibility and integration problem. There's no way to connect raw patient visit records to financial systems, external health benchmarks or analytical tools without a lot of manual work.

This is a direct impact on patient visit data's operational value. Sekarini et al. demonstrated how the implementation of a dashboard in the hospital can help administrators to monitor the trend of hospital admissions, patient flow and make faster decisions without clinical involvement. However, their system used just one Hospital Management Information System as the only source of data, which prevented them expanding patient visit data with Financial Settlement outcomes or comparing with external health standards. This constraint highlights a larger architectural problem the single source dashboard system is unable to solve: one that demands a multi-source data pipeline to ingest, integrate and continuously improve the quality of the data that is collected from patient sources.

B. Data Pipelines and ETL Processes for Patient Health Records

Given the current challenges with manual data processing, the evolution of structured data pipelines into patient health records has become a key research focus. Saraswat et al. [2] proved that an ETL pipeline for Electronic Health Records allows to integrate and transform patient information from multiple sources and provide a consistent view of patient data for subsequent analysis and machine learning models. Their research showed that patient information from structured fields like diagnosis codes and lab results to unstructured fields like clinical notes needs to be systematically captured, cleaned and unified before it has any value for analysis. It's this principle that forms the basis of the design logic for any healthcare data pipeline: no matter how rich a data record is with patient information, it cannot be used to make accurate decisions without being transformed.

When applied to a more modern processing paradigm, Shrotriya et al. [3] showed that Apache Spark is particularly suited for handling the scale and variety of healthcare data in pipeline implementations. They compared the performance of Spark's in-memory distributed computing model with that of traditional data processing models over a large patient population in multiple healthcare use cases and found that Spark's model is much more effective in scenarios involving

large patient datasets, such as electronic health record management, hospital readmission reduction and prediction of patient outcomes. Most importantly, they discovered that Spark's greater integration with machine learning libraries enables the same pipeline to be used for data transformation and predictive analytics in a single, cohesive framework, instead of utilizing separate systems. Saraswat et al. [2] and Shrotriya et al. [3] independently conclude that a modern patient visit data pipeline should be automated and distributed, which are attributes that exclude traditional ETL tools and traditional web-based systems, such as that of Sekarini et al. [1].

C. Data Lakehouse Architecture as the Foundation for Patient Data Analytics

The architecture that stores and organizes patient data will determine if it can be effectively used for analytics, while ETL pipelines will address the movement of patient data. The traditional separation between data lakes for raw storage and data warehouses for structured querying has been identified as a source of inefficiency in healthcare data environments. As a solution to this, Aziz [4] introduced the Open Lakehouse Framework, which shows that by taking the flexibility of a data lake and adding the structured query capabilities of a data warehouse, healthcare organizations can ingest, store and analyze patient data, as well as load it into the analytical system, without having to duplicate infrastructure. The system's structure is particularly suited to integrate Apache Spark as the processing engine, due to its capacity to process all the different formats like structured, semi-structured and unstructured patient visit data from multiple sources.

The relevance of this architectural approach for patient visit monitoring is further supported by the study conducted by Frontiers in Digital Health [5] that found that automated ELT processes with dimensional modeling are the best way to design a healthcare data warehouse. They also determined that the Star Schema can greatly improve the performance of queries for patient-level analytical workloads, allowing hospital administrators to slice and filter data across dimensions

such as date, medical condition, insurance provider and hospital without suffering the performance overheads of traversing big flat datasets. What's most important, is that they have identified the two top areas of failure in patient data warehouse implementations: inconsistent patient identifiers and broken data integration between systems, highlighting the need for a structured layer of cleaning and integration (the Silver layer of the Medallion Architecture) before patient data reaches the analytical Gold layer.

What Aziz [4] and Frontiers [5] have together defined is the need for a patient visit data pipeline to be designed so that raw data ingestion, data quality processing and analytical modelling are separated into different controlled layers. This is the core principle behind the Medallion Architecture used in this project: patient visit data from three sources is progressively refined from raw Bronze Parquet files, to cleaned and integrated Silver datasets, to a Star Schema Gold layer that is loaded into PostgreSQL and exposed to users via Metabase dashboards.

D. AI-Based Anomaly Detection for Patient Billing Records

A major drawback of existing patient visit monitoring systems is that they lack the ability to identify suspicious or abnormal coding in patient records automatically. Sekarini et al. [1] admitted that their dashboard system could show billing information, but did not have a way to identify billing records that seemed to be inconsistent with patients' condition or length of stay. This gap is analytically significant as identified by both Tandfonline [6] and PMC [7], anomalous billing in healthcare patient records is a problem that manual review cannot effectively address at scale.

Unsupervised machine learning methods, such as Isolation Forest, Local Outlier Factor and autoencoder models, are particularly useful for the detection of abnormal billing patterns in healthcare data, as they do not depend on the existence of pre-labeled anomaly data in the real setting of hospitals, as shown by the work of Tandfonline [6]. This result is also supported by PMC [7] that evaluated several unsupervised anomaly

detection algorithms on health care insurance claims and showed that Isolation Forest was able to generate anomaly scores with meaningful health insurance claims insights, isolating those with claims that were generated from very different procedures and diagnosis patterns from the rest of the claims. Importantly, both studies found that the anomaly detection model performs best when it is given the patient's contextual features, not just the amount billed, but the medical condition, length of stay and admission type for each patient visit. This creates a direct relationship between the quality of the data pipeline and the quality of the anomaly scores derived from the model: The higher the quality of the patient visit data that is pushed into the model, the higher the quality of the anomaly scores.

That is why the multi-source integration approach is justified in the project. The Kaggle patient visiting CSV provides billing amount, medical condition and length of stay. The WHO GHO API provides global health benchmarks which can help to understand if a patient's condition and length of stay is normal or abnormal. The PostgreSQL financial database offers insurance claim settlement status which shows if the insurer accepted the billing amount or disputed it. No single-source system identified in literature achieves such a high level of integration in the Silver layer, which is needed to be meaningful for running Isolation Forest anomaly detection at the Gold layer.

III. METHODOLOGY

This project adopts Cross Industry Standard Process for Data Mining (CRISP-DM) as the development methodology in order to achieve the objectives. The framework CRISP-DM was selected because it is a widely used framework for data-driven projects aimed at data collection, preparation, analysis and visualization. CRISP-DM is flexible, unlike the Waterfall model, where developers move to the next phase after finishing the previous one. For example, if problems are found in the patient visit records during the Data Preparation phase, the project can go back to the Data Understanding phase to review and better understand the data. This

iterative approach makes CRISP-DM suitable for a patient visit data pipeline project, where data quality and integration issues are often discovered during the data exploration and processing stages. The six phases of CRISP-DM that are used throughout this project are shown in Figure 1.

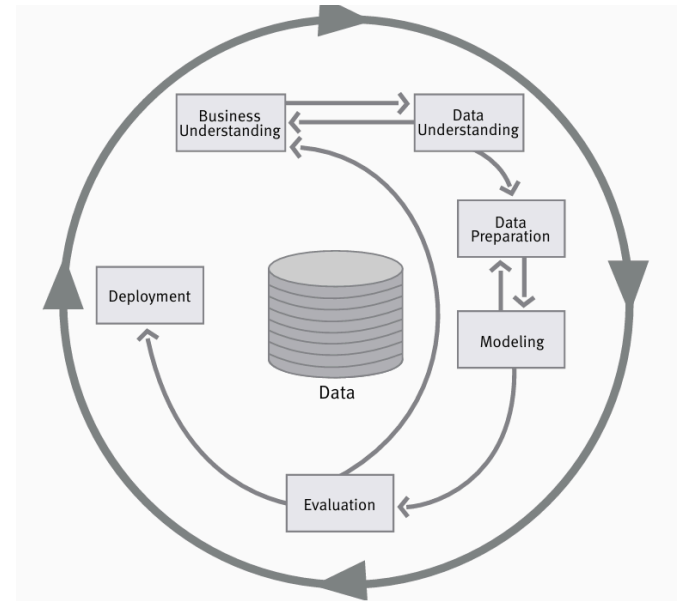


Figure 1 CRISP-DM process model (adapted from Chapman et al., CRISP-DM 1.0: Step-by-Step Data Mining Guide, 2000).

A. Phase 1: Business Understanding

In this phase, the healthcare domain was studied to identify the main problem, which is the difficulty for hospital administrators to track patient visit patterns, monitor billing and detect unusual charges due to scattered and unintegrated data sources. Three data sources were selected to provide a complete view of each patient visit, from admission to treatment and billing. Success was measured by successful data ingestion, proper integration in the Silver layer, correct Star Schema in the Gold layer and working dashboards with anomaly detection results.

B. Phase 2: Data Understanding

This project uses three different data sources to demonstrate the integration of healthcare data from multiple systems.

Data Source 1: Patient Visit Records -Healthcare Dataset (CSV File)

The first dataset used in this project is the Kaggle Healthcare Dataset. It consists of 55,500 records and 15 attributes in a CSV file. Attributes include patient demographics, medical conditions, admission details, hospital information, insurance providers, billing amounts, medications and test results. This dataset provides the foundation of the analytics pipeline and is the main source of patient-level data.

Data Source 2: Patient Condition Benchmarks — WHO Global Health Observatory (GHO) REST API

The second data source is a healthcare REST API that provides hospital-level statistics in JSON format. The API contains information such as hospital occupancy rates, average length of stay and disease prevalence statistics. The data is retrieved using HTTP requests and loaded into Apache Spark for processing. This source enables comparisons between patient information and more common medical trends and provides additional healthcare context.

The project is based on data from the API: <https://ghoapi.azureedge.net/api/NCDMORT3070>, providing JSON data for country and gender specific disease mortality rates and health indicators at the global level.

Data Source 3: Patient Billing Settlement — Hospital Financial Records (PostgreSQL Database)

The third data source is a PostgreSQL database that contains hospital financial and operational information. The database stores records such as department budgets, insurance claim statuses and cost benchmarks for medical procedures. The data is imported into Spark using a JDBC connection. This source is important because it allows patient billing information to be linked with financial performance data and supports more detailed analysis in the Gold layer.

C. Phase 3: Data Preparation

The third phase focuses on collecting, cleaning, transforming and combining data from the three data sources. The goal is to convert the raw patient visit data into a single clean, complete and well-organized dataset that is ready for analysis and modelling. This project uses the Medallion Architecture where data is set up in Bronze, Silver and Gold layers with Extract, Load, Transform (ELT). Since the project integrates data from multiple sources with different formats and structures, separating the data into layers makes the pipeline easier to manage and maintain.

The ELT approach is used because Apache Spark can efficiently process large amounts of data. Instead of transforming data before loading it, the raw data is first loaded into the Bronze layer and then transformed in later stages. This approach preserves the original data and makes it easier to reprocess the data if business requirements change in the future.

Bronze Layer — Patient Visit Data Ingestion

In Bronze layer, data from all three sources is loaded into Apache Spark and stored in Parquet format without changing the original content. This ensures that all source data is preserved and can be traced back if needed.

For the CSV dataset, Spark reads the file directly using the CSV reader. For the REST API, JSON responses are retrieved and converted into Spark DataFrames. For the PostgreSQL database, Spark connects through JDBC and imports the required tables.

Metadata columns like ingestion timestamp and source name are added during ingestion to support data lineage and tracking. Column names are also converted to snake_case for consistency. The resulting datasets are saved as Parquet files to the Bronze layer.

Silver Layer — Patient Visit Data Cleaning and Integration

The Silver layer consists of cleaning, validating and preparing the data for analysis. For the patient dataset, data types are converted into appropriate formats. For

example, dates are converted to date format, billing amounts to decimal format and age to integer format. Duplicate records are removed, text values are standardized and a new column called 'length_of_stay_days' is calculated using the admission and discharge dates. A unique patient identifier is also generated.

For the API data, nested JSON fields are flattened into tabular columns. Missing values are handled and hospital identifiers are standardized to match the patient dataset.

For the PostgreSQL financial data, data inconsistencies are resolved and currency values are standardized. Hospital identifiers are also aligned with the other datasets.

The three datasets are merged after cleaning, based on common fields such as hospital name and insurance provider. The result is a single healthcare dataset that combines patient, operational and financial information. The Silver layer produces a single enriched patient visit dataset. Each row represents one complete patient visit and includes the original clinical and patient information from the CSV dataset, together with global health data from the WHO API and insurance claim information from the PostgreSQL database. The final dataset is stored as a Parquet file and is used as the input for Phase 4, which focuses on data modelling.

D. Phase 4: Modelling

The fourth phase transforms the unified dataset from the Silver layer into an analytical model that is easier to query and use for reporting. In this phase, the AI anomaly detection model is also applied to identify unusual billing amounts. The result is an enriched dataset where each patient visit record includes a billing anomaly indicator for further analysis.

Gold Layer — Patient Visit Star Schema Modelling

The Gold layer contains data that is ready for reporting and business analysis. The cleaned Silver layer data is transformed into a Star Schema to improve query performance and simplify reporting. Several dimension

tables are created, including 'dim_patient', 'dim_date', 'dim_hospital', 'dim_doctor', 'dim_medical_condition', 'dim_insurance_provider', 'dim_medication'. They are connected to a 'fact_patient_visit' table that stores billing amount, length of stay, room number, admission type, test results and insurance claim status.

A denormalized analytical view is also created to support faster dashboard queries. The Gold layer is loaded into PostgreSQL using Spark's JDBC connector and serves as the data source for Metabase dashboards.

AI Algorithm Integration — Patient Billing Anomaly Detection

During the Silver-to-Gold transition stage, a predictive billing anomaly detection model is added to the data pipeline. The pipeline uses Spark MLlib's Isolation Forest to detect patient billing records that are significantly different from expected given the patient's medical condition, length of stay and insurance provider. This creates an 'anomaly_flag' column to the Gold layer fact table which is shown in the Metabase dashboard to alert administrators to potentially erroneous or fraudulent billing entries. The model is trained on cleaned Silver layer data and is used as a batch inference step before loading the Gold layer.

This algorithm was chosen because the healthcare billing data does not have any pre-labeled fraud or anomaly data, which makes supervised learning unsuitable. Moreover, Isolation Forest is efficient to work with large-scale data in the distributed Spark environment.

The AI anomaly detection model uses several patient visit features that may affect the billing amount. These include 'billing_amount', 'length_of_stay_days', 'age', 'medical_condition' and 'admission_type'. Since 'medical_condition' and 'admission_type' are text values, they are first converted into numeric values using 'StringIndexer'. All the features are then combined using 'VectorAssembler' before being used to train the Isolation Forest model.

The model uses a contamination value of 0.05, which means about 5% of the patient visit records are expected to be identified as billing anomalies. This setting is

suitable for detecting a small number of unusual billing records while treating most records as normal.

E. Phase 5: Evaluation

The fifth phase determines if the patient visit data pipeline generates accurate, complete and useful results before deployment. The evaluation is carried out in three areas.

Data Quality Evaluation validates that all patient visit records are being successfully imported by comparing the count of patient visit data in the Bronze layer with the original data source. It also guarantees that there are no missing values in the important fields after the Silver layer transformation, like 'billing_amount', 'date_of_admission' and 'hospital_name'. Furthermore, the value of length_of_stay_days is checked for being > 0 and duplicate patient_visit records are eliminated from the final Silver dataset.

The Star Schema Evaluation verifies that the Gold layer has been built correctly. It verifies that all foreign keys in the fact_patient_visit table are matching the corresponding records in the dimension tables, verifies that there are no orphan records and verifies that the number of the records in the Gold layer is equal to the number of records in the unified Silver dataset.

The performance of the anomaly detection model is evaluated using AI Model Evaluation. The number of records identified as anomalies is compared to the set contamination percentage of 5%. The flagged records are also checked to ensure that they are flagged as being a high billing amount for a patient with similar medical conditions and length of stay, thus demonstrating that the model is identifying real billing anomalies.

F. Phase 6: Deployment

The sixth phase focuses on deploying the complete patient visit data pipeline and dashboard so that users can access the results.

The system is deployed with Docker, each container running one of the three components: Apache Spark, PostgreSQL and Metabase. It is within these containers that all data ingestion, processing, AI anomaly detection,

storage and visualization tasks are executed. Services are managed by Docker Compose.

The deployment process is fixed. First, the Bronze layer imports data from the three data sources. This is followed by the Silver layer that cleans, transforms and aggregates the data into a single patient visit dataset. Next, the Isolation Forest algorithm picks out the patient visits that have odd dollar amounts billed. Data is loaded to the Gold layer star schema in PostgreSQL using JDBC. Lastly, Metabase is integrated with the Gold database, and the dashboard is shared with users for exploring the patient visit data at 'http://localhost:3000.'

IV. SYSTEM DESIGN & ARCHITECTURE

A. System Design

The proposed system uses an Extract, Load, Transform (ELT) approach along with Medallion Architecture. The data pipeline begins by collecting healthcare data from three different sources: a CSV file, a REST API and a PostgreSQL database. Apache Spark is used to ingest data from these sources and store it in the Bronze layer as raw Parquet files. Then, data ingestion, data processing, AI enhancement, data storage and data visualization run inside Docker containers. Figure 2 shows the data architecture.

B. Use Case Diagram

Figure 3 shows the use case diagram for the healthcare data pipeline system.

C. Database Design

Figure 3 shows the source entity relationship diagram (ERD) that is connected from three data sources. Figure 4 shows the star schema for analysis.

V. IMPLEMENTATION

A. Pipeline Execution

The patient visit data pipeline was developed using PySpark 3.5.1 on a Windows computer running in local[*] mode. Docker was used to set up the working environment, which included PostgreSQL 15, Apache Spark 3.5.1 and Metabase v0.49.6. The entire pipeline was automated using the 'run_pipeline.py' script. This

script is designed to run five python scripts one after another with Python's subprocess module, making sure that each script runs only after the previous one has finished.

B. Bronze Layer Results

The Bronze layer successfully collected data from three different sources and stored them as Parquet files shown in Table 1.

Table 1 Bronze Layer Results

Source	Records Ingested	Format	Storage Path
Kaggle Healthcare CSV	55,500	CSV → Parquet	bronze/kaggle/
WHO GHO API	12,936	JSON → Parquet	bronze/who_api/
PostgreSQL Financial	500	JDBC → Parquet	bronze/postgres/

Two metadata columns, `'_ingested_at'` and `'_source_name'`, were also added to every record to record when the data was loaded and where it came from.

C. Silver Layer Results

The Silver layer focused on cleaning and transforming the data before combining it for analysis. For the Kaggle patient visit dataset, date columns were converted into the correct date format, while `'billing_amount'` was converted into a decimal data type. The `'age'` and `'room_number'` columns were converted into integer values. Medical condition names were standardized using the `'initcap()'` function, and a new column named `'length_of_stay_days'` was created by calculating the difference between the admission date and discharge date.

Records that contained invalid values such as length of stay of zero or less and billing amount of zero or less were removed. After cleaning and removing duplicate records, the dataset contained 54,860 patient records.

The cleaned dataset was then joined with the PostgreSQL financial dataset using `'hospital_name'` and

`'insurance_provider'` as the matching fields. The dataset had 55,370 records after the join. The increase in records occurred because some hospitals and insurance providers had multiple matching financial records.

The WHO GHO API dataset was not joined directly with the patient data because it contains country-level health statistics rather than individual patient information. Instead, it was kept as a separate reference dataset for future analysis.

D. Gold Layer Results

The cleaned Silver dataset was transformed into a Star Schema and stored in the PostgreSQL `'gold_db'` database. Before creating the Star Schema, the Isolation Forest algorithm from the scikit-learn library was applied to identify unusual patient records. Using a contamination value of 0.05, the model detected 2,769 patient visits, which is about 5% of the dataset, as anomalies.

The Gold layer consists of seven dimension tables and one fact table. During implementation, it was found that the `'dim_hospital'` and `'dim_doctor'` tables contained more records than expected because the financial data join created duplicate entries. In a real production system, these tables would be further cleaned by removing duplicates and using surrogate keys. However, for this project, natural keys such as `'hospital_name'`, `'doctor'`, `'medical_condition'`, `'insurance_provider'` and `'medication'` were used to maintain the relationships between the fact and dimension tables.

E. AI Anomaly Detection Results

The anomaly detection model was built using the Isolation Forest algorithm from the scikit-learn library because PySpark 3.5.1 does not provide a built-in Isolation Forest model. Four numerical features were used to train the model: `'billing_amount'`, `'length_of_stay_days'`, `'age'` and `'room_number'`.

The Silver dataset was first converted into a pandas DataFrame for model training and prediction. After prediction, the anomaly results were added back to the Spark DataFrame before loading the data into the Gold layer.

The model identified 2,769 records (5.04%) as anomalies. Most of these records had much higher billing amounts than other patients with similar medical conditions and lengths of stay. This may indicate possible billing errors or unusual transactions that require further investigation.

F. Testing

Testing was carried out using both Black Box and White Box approaches to ensure the patient visit data pipeline was correct and complete.

Black Box Testing

Black box testing focused on checking the pipeline based on inputs and outputs without looking at the internal code or processing steps. Table 2 shows each stage of the pipeline was tested to confirm that it produced the expected output when given specific input data.

Table 2 Test Cases for Black Box Testing

Test Case	Input	Expected Output	Result
CSV ingestion row count	healthcare.csv (55,500 rows)	55,500 rows in Bronze layer	Pass
WHO API ingestion	API endpoint call	12,936 rows in Bronze layer	Pass
PostgreSQL ingestion	hospital_financial table (500 rows)	500 rows in Bronze layer	Pass
Anomaly detection output	Silver dataset (55,370 rows)	5% flagged ($\approx$ 2,769 records)	Pass
Metabase database connection	gold_db credentials	Successful connection	Pass

Dashboard page loading	4 dashboard pages	All pages render without error	Pass
------------------------	-------------------	--------------------------------	------

White Box Testing

White box testing examines the internal logic of the Silver layer to ensure that all data transformations and data quality rules were correctly implemented in the code. Table 3 shows the test cases.

Table 3 Test Cases for White Box Testing

Test Case	Code Used	Expected Result	Result
Null values in billing_amount	<code>df.filter(col("billing_amount").isNull()).count()</code>	0 nulls	Pass
Null values in date_of_admission	<code>df.filter(col("date_of_admission").isNull()).count()</code>	0 nulls	Pass
Invalid length of stay	<code>df.filter(col("length_of_stay_days") <= 0).count()</code>	0 records	Pass
Duplicate records	<code>df.count() vs df.distinct().count()</code>	No duplicates	Pass
Billing amount range	<code>df.filter(col("billing_amount") <= 0).count()</code>	0 records	Pass

VI. RESULT AND DISCUSSION

The patient visit data pipeline successfully processed a total of 59,724 patient visit records and presented the results through four interactive Metabase dashboard

pages. The dashboards provide an overview of patient visits, financial performance, hospital operations and billing anomalies.

The Patient Visit Overview dashboard in Figure 6 shows 59,724 total visits, with an average billing amount of RM25,650.83 and an average length of stay of 15.6 days. It also identified 3,031 billing records as anomalies. Patient admissions are almost evenly distributed among Emergency, Elective and Urgent categories, while all six medical conditions recorded similar numbers of visits. Monthly admissions remained stable from 2019 to 2024, indicating consistent hospital activity.

The Billing and Financial Analysis dashboard in Figure 7 shows that average billing amounts are similar across all medical conditions, ranging from approximately RM25,000 to RM26,000. However, claim status results indicate that only 31.4% of claims have been settled, while the remaining claims are either pending or rejected. This suggests that claim processing should be improved to reduce financial risk.

The Hospital and Length of Stay Analysis dashboard in Figure 8 shows that patients stayed in the hospital for an average of about 15 days regardless of medical condition or admission type. Test results were also evenly distributed across normal, abnormal and inconclusive categories.

Finally, the Anomaly Alert Dashboard in Figure 9 identified a 5.08% anomaly rate, with 3,031 patient billing records flagged for further review. These records had much higher billing amounts than normal cases, showing that the anomaly detection model was effective in identifying unusual billing patterns.

VII. CONCLUSION

This project successfully developed an end-to-end patient visit data pipeline using Apache Spark and the Data Lakehouse architecture. The pipeline collected data from three different sources, processed it through the Bronze, Silver and Gold layers, applied Isolation Forest for anomaly detection and displayed the results in a PostgreSQL and Metabase dashboard.

The pipeline successfully processed 55,500 patient visit records, integrated financial data, detected 2,769 anomalous billing records and stored the final data in a Star Schema for analysis. The results show that the pipeline can help hospital administrators monitor patient visits, analyze billing performance and identify unusual billing records.

Several lessons were learned during this project, including the importance of handling one-to-many joins correctly and choosing suitable tools for AI model implementation. The Medallion Architecture also proved effective in organizing and managing the data pipeline.

In the future, the project can be improved by using surrogate keys in the Star Schema, applying more advanced anomaly detection methods, integrating the WHO GHO API data more effectively, and automating the pipeline with Apache Airflow for scheduled data processing.

REFERENCES

- [1] I. G. A. A. Sekarini, A. A. S. Widhiyanti, and N. M. R. M. Dewi, "Dashboard Application For Monitoring Patient Data In Hospitals," *Eduvest — Journal of Universal Studies*, vol. 5, no. 8, pp. 10339–10352, Aug. 2025. <https://eduvest.greenvest.co.id/index.php/edv/article/view/51010/4441>
- [2] B. K. Saraswat, N. Varshney, and P. C. Vashist, "Development of ETL pipeline for Electronic Health Record to support Machine Learning based Approaches for Security and Prediction," *International Journal of Intelligent Systems and Applications in Engineering*, vol. 12, no. 19s, pp. 168–189, 2024. <https://ijisae.org/index.php/IJISAE/article/view/5057>
- [3] L. Shrotriya, K. Sharma, D. Parashar, K. Mishra, S. S. Rawat, and H. Pagare, "Apache Spark in Healthcare: Advancing Data-Driven Innovations and Better Patient Care," *International Journal of Advanced Computer Science and Applications*, vol. 14, no. 6, pp. 608–616, 2023. doi: 10.14569/IJACSA.2023.0140665. <https://thesai.org/Publications/ViewPaper?Volume=14&Issue=6&Code=IJACSA&SerialNo=65>

- [4] F. Aziz, "Next-Generation Healthcare Analytics: The Open Lakehouse Framework," *IJRAR — International Journal of Research and Analytical Reviews*, vol. 11, no. 2, pp. 94–105, May 2024. doi: 10.2139/ssrn.5065660. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5065660
- [5] "Building a Healthcare Data Warehouse: Considerations, Opportunities, and Challenges," *Frontiers in Digital Health*, 2025. <https://www.frontiersin.org/journals/digital-health/articles/10.3389/fdgth.2025.1691142/full>
- [6] "Enhancing Healthcare Data Integrity: Fraud Detection Using Unsupervised Learning Techniques," *International Journal of Computers and Applications*, Taylor & Francis, 2024. <https://www.tandfonline.com/doi/full/10.1080/1206212X.2024.2408262>
- [7] "Healthcare Insurance Fraud Detection Using Data Mining," *BMC Medical Informatics and Decision Making*, PMC, 2024. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11046758/>
- [8] CRISP-DM process model (adapted from Chapman et al., *CRISP-DM 1.0: Step-by-Step Data Mining Guide*, 2000). <https://www.dataprix.com/files/CRISP-DM.pdf>

APPENDIX

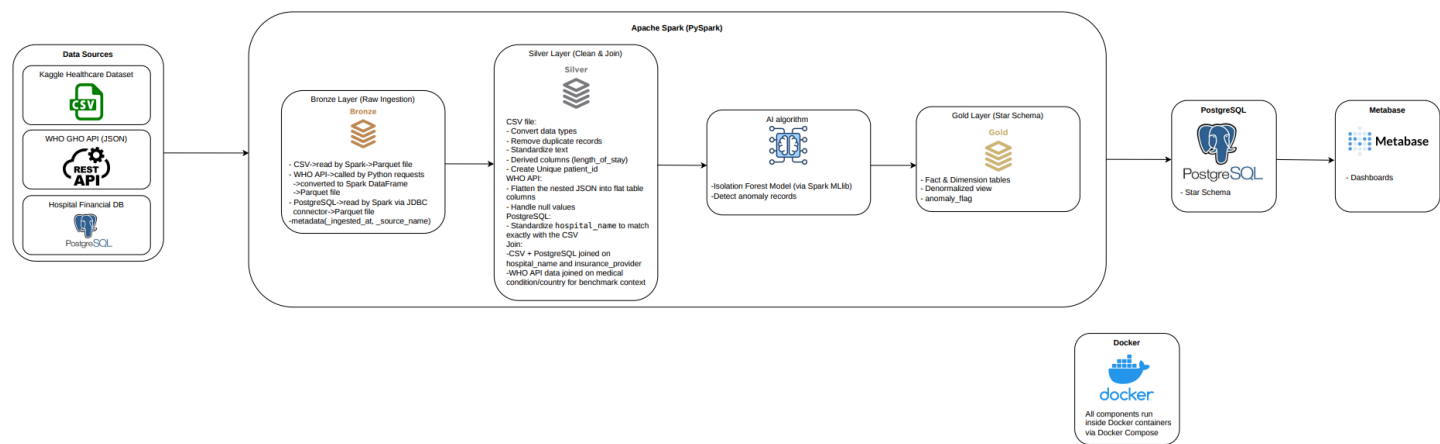


Figure 2 Data Architecture

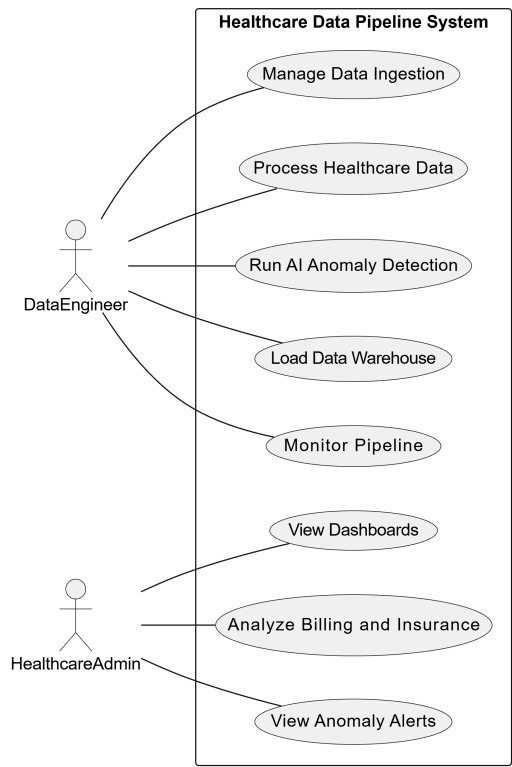


Figure 3 Use Case Diagram

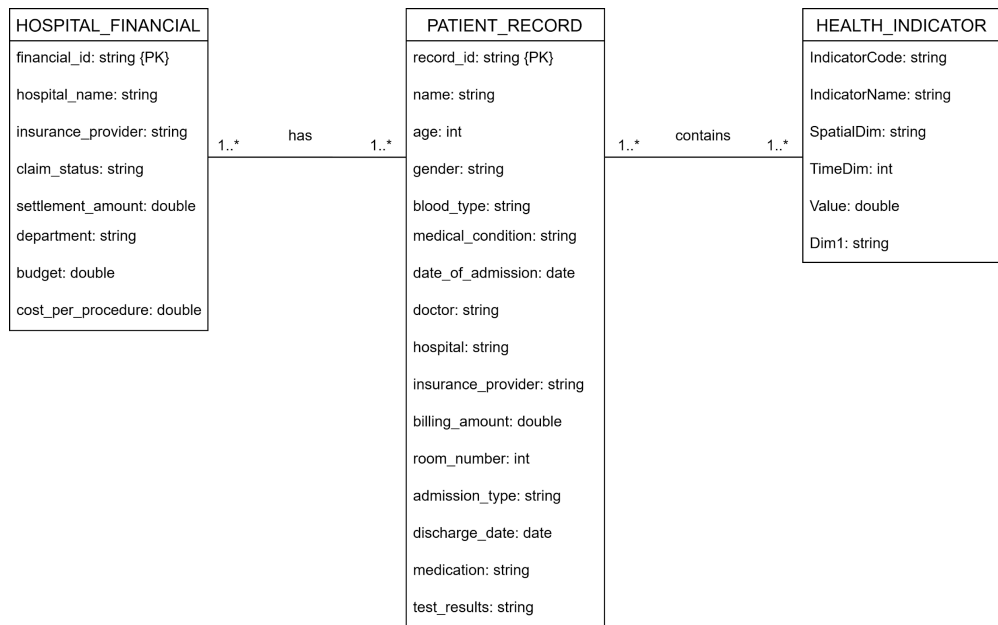


Figure 4 Entity Relationship Diagram (ERD)

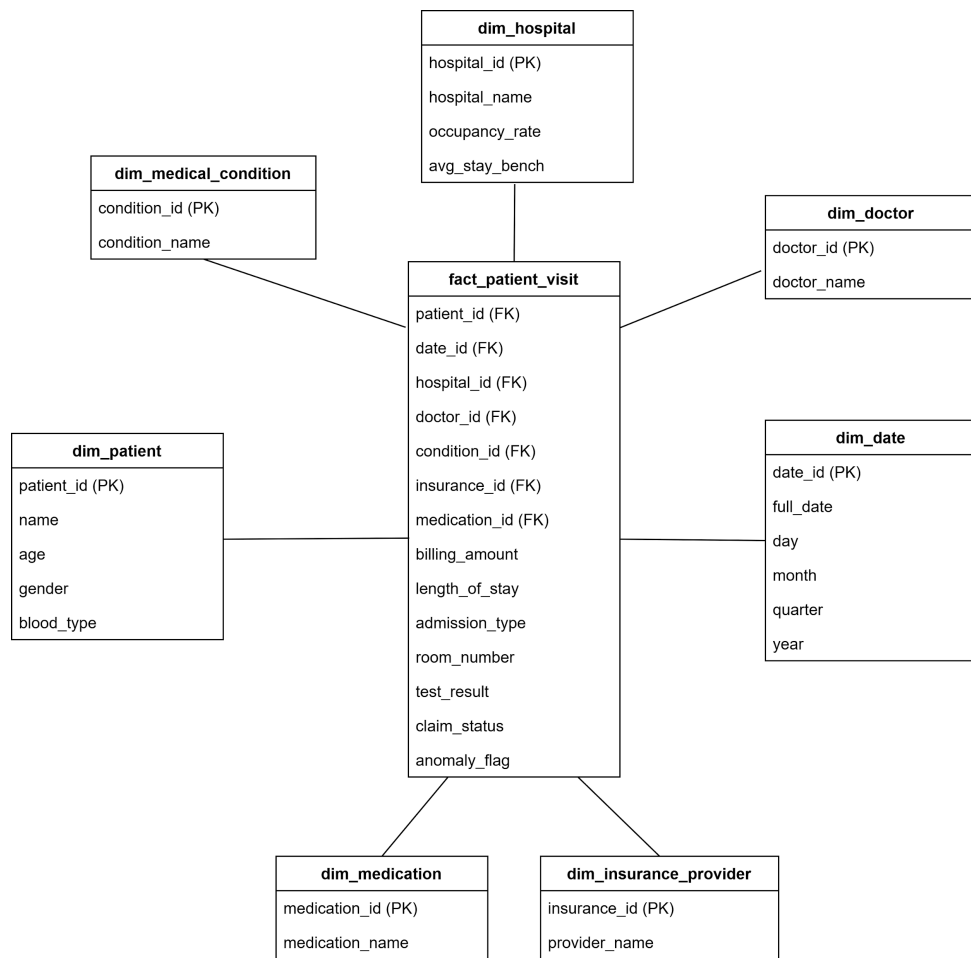


Figure 5 Star Schema

Dashboard Patients Visit

Patient Visit Overview

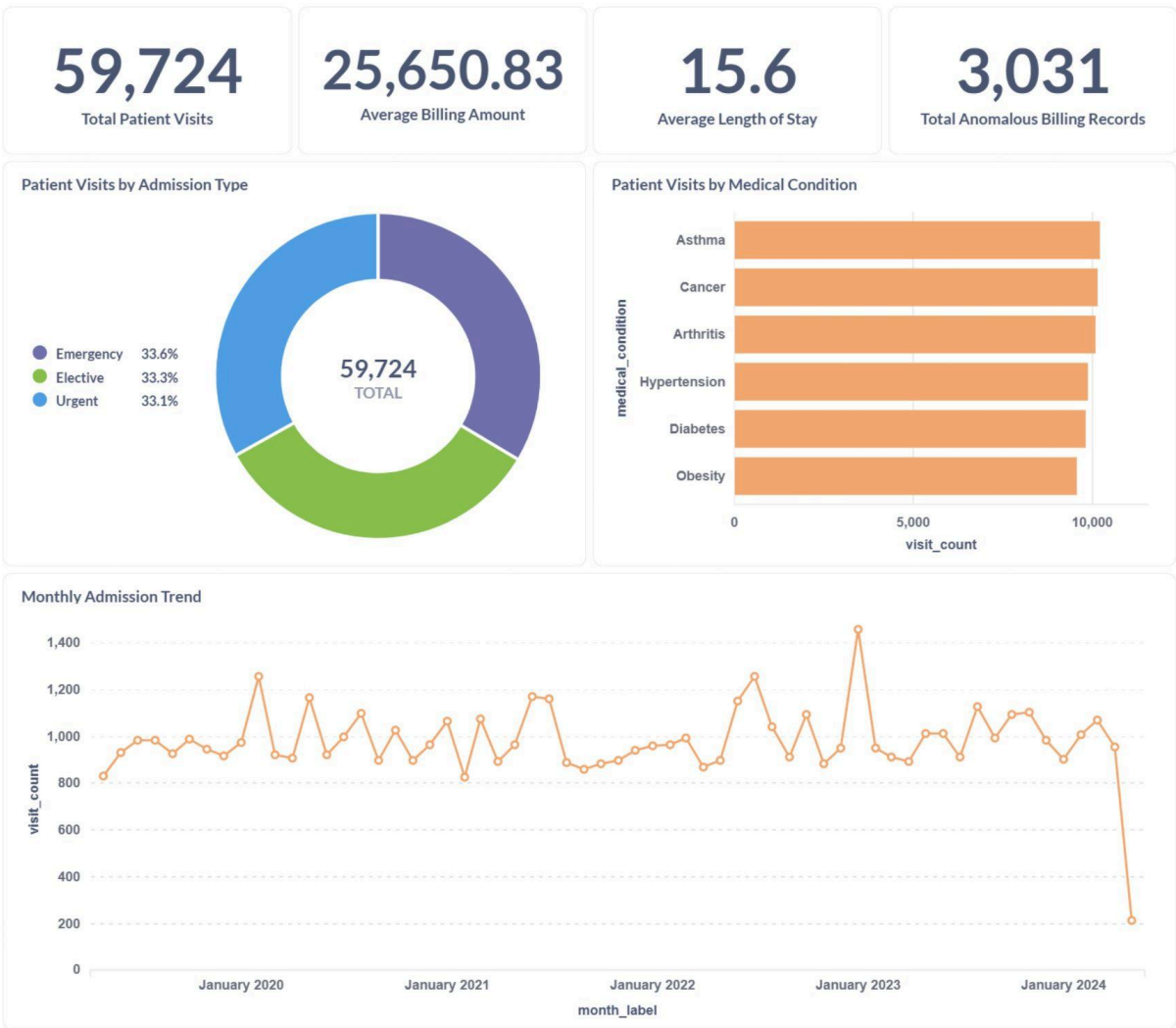


Figure 6 Dashboard-Patient Visit Overview

Billing and Financial Analysis



Figure 7 Dashboard-Billing and Financial Analysis

Hospital and Length of Stay Analysis

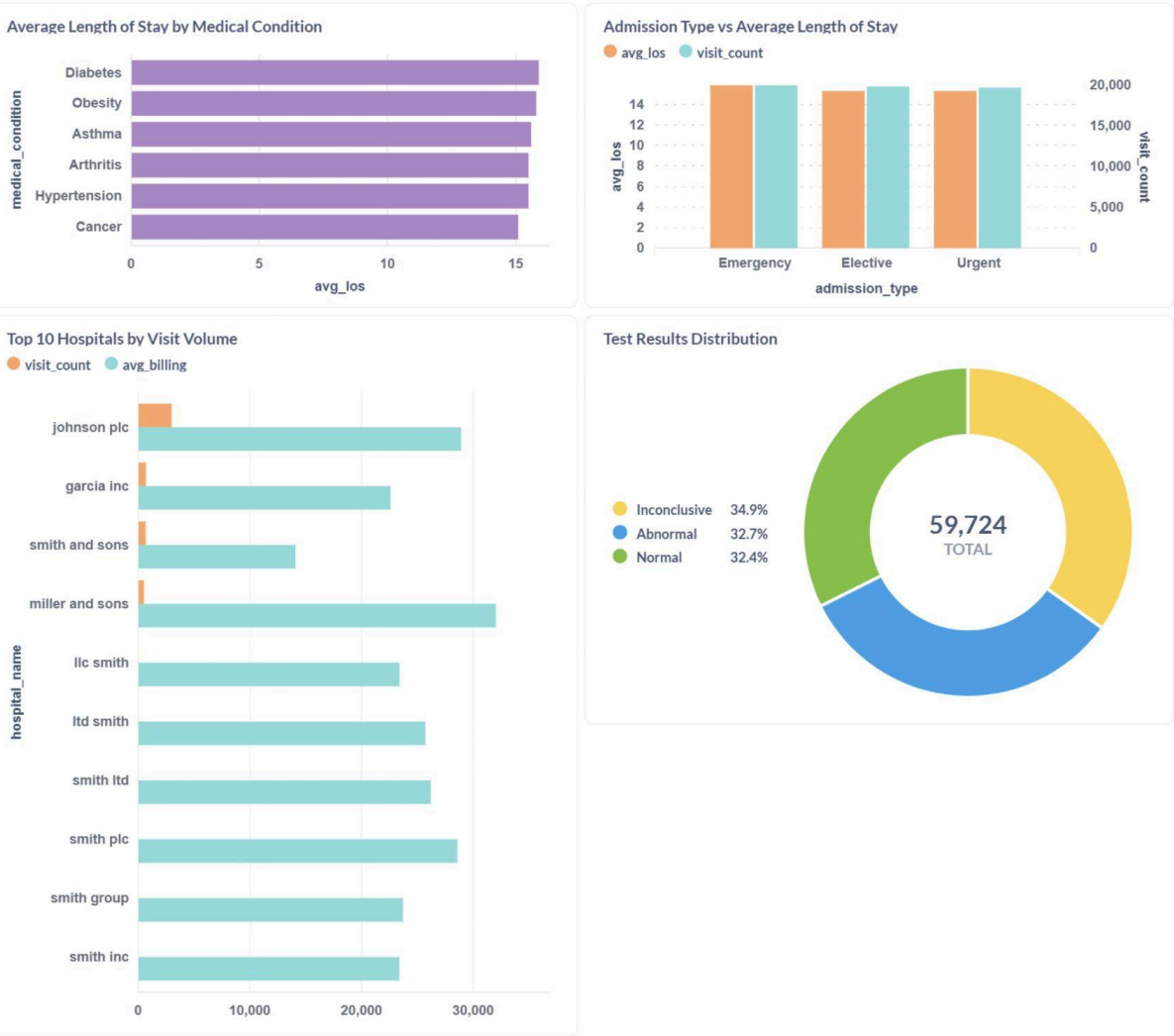


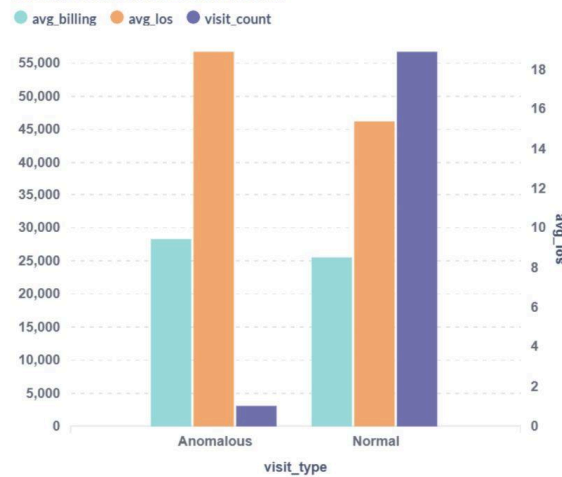
Figure 8 Dashboard-Hospital and Length of Stay Analysis

Anomaly Alert Dashboard

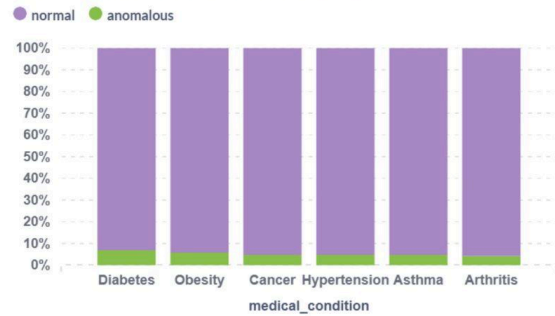
5.08%

Anomaly Rate

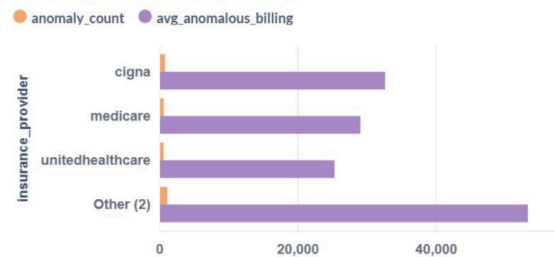
Average Billing: Anomalous vs Normal



Anomalous vs Normal Visits by Medical Condition



Anomalous Billing by Insurance Provider



Flagged Patient Visit Detail Table

visit_id	hospital_name	medical_condition	admission_type	billing_amount	length_of_stay_days	insurance_provider	anomaly
874	miller ltd	Diabetes	Urgent	52,211.85	1	aetna	
17,179,870,703	rocha-carter	Cancer	Emergency	52,092.67	2	medicare	
17,179,874,051	briggs walker martinez, and	Obesity	Elective	52,024.73	26	cigna	
34,359,743,265	and small stephens harrington,	Arthritis	Urgent	51,975.97	27	blue cross	
42,949,678,002	and small stephens harrington,	Arthritis	Urgent	51,975.97	27	blue cross	

Rows 1-5 of 100 < >

Figure 9 Dashboard-Anomaly Alert Dashboard

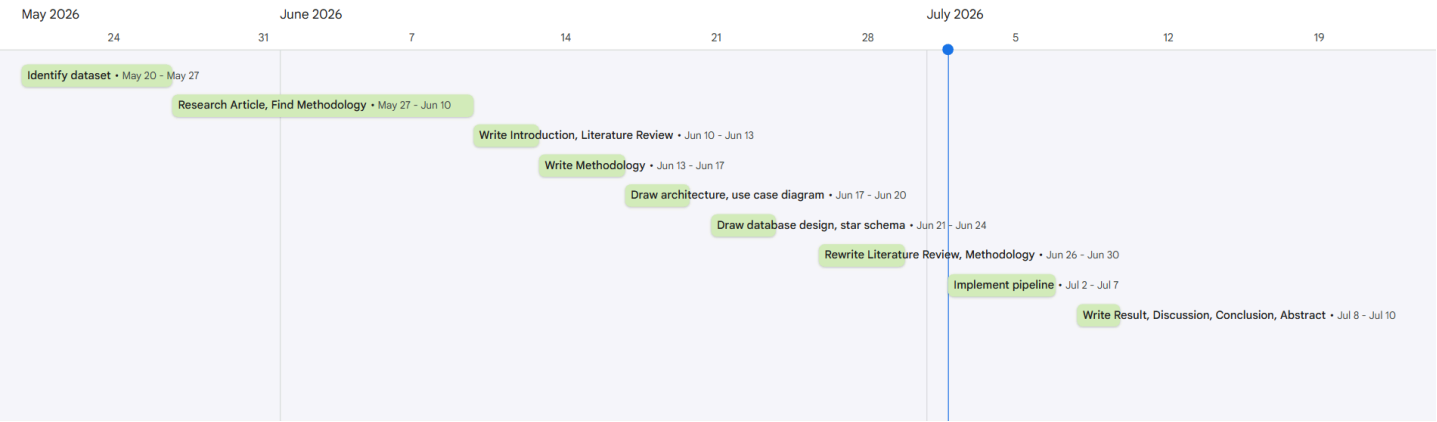


Figure 10 Project Timeline